

Polyfills and Transpilers – Simple Hinglish Summary

Source: <https://javascript.info/polyfills>

Main baat

JavaScript **lagatar evolve** hoti rehti hai. Naye proposals aate hain, analyze hote hain, aur theek lage to official **specification** me jud jate hain.

Lekin JavaScript engines banane wali teams ki **apni priorities** hoti hain. Wo kabhi draft wale proposals pehle implement kar deti hain, aur spec me pehle se maujood kuch cheezein baad ke liye chhod deti hain (kam interesting ya mushkil hone ki wajah se).

Isliye **engine ka standard ka sirf ek hissa implement karna common hai**.

Current support dekhne ke liye: <https://compat-table.github.io/compat-table/es6/>

Problem

Hum programmers **naye features use** karna chahte hain. Lekin naya code **purane engines** par kaise chalega jo naye features nahi samajhte?

Iske liye do tools hain:

1. Transpilers
2. Polyfills

1. Transpilers

Transpiler ek software hai jo **ek source code ko dusre source code me translate** karta hai. Wo modern code ko padhkar (parse karke) use **purane syntax** me dobara likh deta hai, taaki purane engines me bhi chale.

Example: 2020 se pehle JavaScript me `(??)` (nullish coalescing) nahi tha. Purana browser `height = height ?? 100` samajh nahi paayega.

Transpiler `height ?? 100` ko `(height !== undefined && height !== null) ? height : 100` me badal deta hai:

```
javascript

// transpiler chalane se pehle
height = height ?? 100;

// transpiler chalane ke baad
height = (height !== undefined && height !== null) ? height : 100;
```

Ab ye code purane engines ke liye suitable hai.

- Aam taur par developer transpiler **apne computer par** chalata hai, aur phir transpiled code server par deploy karta hai.
- **Babel** sabse prominent transpilers me se ek hai.
- Modern build systems (jaise **webpack**) har code change par transpiler **automatically** chala dete hain, isliye ise development me jodna aasan hai.

2. Polyfills

Nayi language features me sirf syntax aur operators nahi, **built-in functions** bhi hote hain.

Example: `Math.trunc(n)` number ka decimal hissa kaat deta hai. `Math.trunc(1.23) = 1`.

Kuch bahut purane engines me `Math.trunc` hota hi nahi, to aisa code fail ho jata hai.

Yahan nayi **functions** ki baat hai, syntax ki nahi, isliye **transpile karne ki zarurat nahi**. Bas gayab function ko **declare** karna hai.

Jo script naye functions add ya update karti hai use "polyfill" kehte hain. Wo gap ko "fill in" karti hai aur missing implementations jodti hai.

`Math.trunc` ka polyfill:

```
javascript

if (!Math.trunc) { // agar ye function nahi hai
  // to implement karo
  Math.trunc = function(number) {
    // Math.ceil aur Math.floor bahut purane engines me bhi hain
    return number < 0 ? Math.ceil(number) : Math.floor(number);
  };
}
```

JavaScript bahut **dynamic** language hai. Scripts kisi bhi function ko add/modify kar sakti hain, built-in functions ko bhi.

Ek interesting polyfill library **core-js** hai. Ye bahut saare features support karti hai aur sirf wahi include karne deti hai jo chahiye.

Summary

Is chapter ka maqsad tumhe **modern aur "bleeding-edge" features** seekhne ke liye pririt karna hai, chahe wo abhi engines me achhi tarah supported na ho.

Bas ye na bhoolo:

- **Transpiler** use karo (agar naya **syntax ya operators** use kar rahe ho)
- **Polyfills** use karo (jo **functions** missing ho sakte hain unke liye)

Isse code sab jagah chalega.

Baad me JavaScript aane par **webpack + babel-loader** plugin se build system bana sakte ho.

Support check karne ke achhe resources:

- <https://compat-table.github.io/compat-table/es6/> - pure JavaScript ke liye
- <https://caniuse.com/> - browser-related functions ke liye

P.S. Google Chrome aam taur par language features me sabse up-to-date hota hai. Tutorial ka demo na chale to Chrome try karo.

Quick Comparison

Baat	Transpiler	Polyfill
Kis liye	Naya syntax/operators (jaise <code>??</code>)	Naye built-in functions (jaise <code>Math.trunc</code>)
Kaam	Code ko purane syntax me rewrite karta hai	Missing function ko add karta hai
Kab chalta hai	Build time par (developer ke computer par)	Runtime par (script ke roop me)
Example tool	Babel (webpack ke saath)	core-js

Yaad rakho:

- Naya **syntax** → Transpiler
- Naya **function** → Polyfill
- Support check karne ke liye: **compat-table** aur **caniuse.com**